Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales

Carrera de Informática

LABORATORIO NRO # 1

Apellido y Nombre: Mamani Challapa Romulo Mateo

CI: 9951413

Carrera: Informática

Materia: Programación 2

Paralelo: “D”

Docente: Lic. Jhonny Roberto Felipez Andrade

Fecha de Entrega: 2 de marzo del 2026

1. *Cronómetro*. Diseñe una clase llamada *Cronometro*. La clase contiene:

   a) Los atributos privados *inicia* y *finaliza* con métodos getter.

   b) Un constructor sin argumentos que inicialica *inicia* con la hora actual.

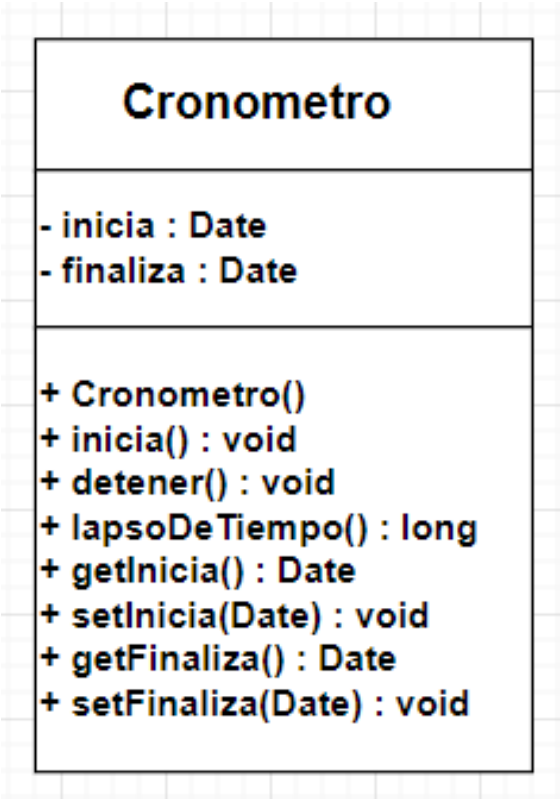
   c) Un método llamado **inicia()** que restablece *inicia* a la hora actual.

   d) Un método llamado **detener()** que establece *finaliza* a la hora actual.

   e) Un método llamado **lapsoDeTiempo()** que retorna el tiempo transcurrido del cronómetro en milisegundos.

   Dibuje el diagrama UML de la clase e impleméntela. Escriba un programa de prueba que mida el tiempo de ejecución de la ordenación de 100.000 números mediante la ordenación por selección.

//DIAGRAMA UML



```
//CODIGO JAVA
package cronos;
import java.util.Date;
import java.text.SimpleDateFormat;

class Cronometro {
        //atributos privados
        //a) Los atributos privados inicia y finaliza con metodos getter
        private Date inicia;
        private Date finaliza;

        //b) Un constructor sin argumentos que inicialica inicia con la hora actual

        public Cronometro() {
                this.inicia = new Date();
        }

    // Comportamiento (Método)
    public void imprimirHora() {
        SimpleDateFormat sdf = new SimpleDateFormat("HH:mm:ss");
        System.out.println("Hora con Date: " + sdf.format(inicia));
    }

    //c) Un metodo llamado inicia() que restablece inicia a la hora actual.

    public void inicia() {
        this.inicia = new Date();
    }

    //d) Un metodo llamado detener() que establece finaliza a la hora actual.
```

```
    public void detener() {
          this.finaliza = new Date();
    }
```

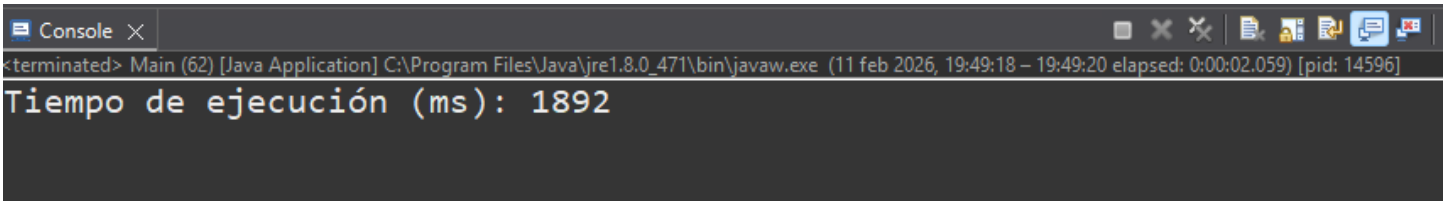
//e) Un metodo llamado lapsoDeTiempo() que retorna el tiempo transcurrido del cronometro en milisegundos.

```
    public long lapsoDeTiempo() {
        if (inicia == null || finaliza == null) {
          return 0;
       }
       return finaliza.getTime() - inicia.getTime();
    }

        //getters y setters
        public Date getInicia() {
                return inicia;
        }
        public void setInicia(Date inicia) {
                this.inicia = inicia;
        }
        public Date getFinaliza() {
                return finaliza;
        }
        public void setFinaliza(Date finaliza) {
                this.finaliza = finaliza;
        }

}
```

```
//MAIN
package cronos;
import java.util.Random;

public class Main {

    public static void main(String[] args) {

        int n = 100000;
        int[] numeros = new int[n];
        Random random = new Random();

        // 1. Llenar el arreglo con números aleatorios
        for (int i = 0; i < n; i++) {
            numeros[i] = random.nextInt(1000000);
        }

        // 2. Crear el cronómetro
        Cronometro cron = new Cronometro();

        // 3. Iniciar medición
        cron.inicia();

        // 4. Ordenar con selección
        ordenacionSeleccion(numeros);

        // 5. Detener medición
        cron.detener();
```

```
        // 6. Mostrar resultado
        System.out.println("Tiempo de ejecución (ms): " + cron.lapsoDeTiempo());
    }

    // Método de ordenación por selección
    public static void ordenacionSeleccion(int[] arr) {

        for (int i = 0; i < arr.length - 1; i++) {

            int indiceMinimo = i;

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[indiceMinimo]) {
                    indiceMinimo = j;
                }
            }

            // Intercambiar
            int aux = arr[i];
            arr[i] = arr[indiceMinimo];
            arr[indiceMinimo] = aux;
        }
    }
}
```

//EJECUCION



2. *Algebra: ecuación lineal 2 x 2.* Diseñe la clase *EcuacionLineal* para un sistema de ecuaciones de 2 x 2.

$$ax + by = e$$

$$cx + dy = f$$

$$x = \frac{ed - bf}{ad - bc} \qquad y = \frac{af - ec}{ad - bc}$$

La clase contiene:

a) Los atributos privados *a*, *b*, *c*, *d* y *f*.

b) Un constructor para los argumentos para *a*, *b*, *c*, *d* y *f*.

c) Un método **tieneSolucion()** que devuelve true si $ad - bc$ no es cero.

d) Dos métodos **getX()** y **getY()** que retornan la solución de la ecuación.

Dibuje el diagrama UML de la clase e impleméntela. Escriba un programa de prueba (Test) que solicite al usuario ingresar a, b, c, d, e y f y muestre el resultado. Si $ad - bc$ es 0, indique que "La ecuación no tiene solución".

Ejemplo de entrada

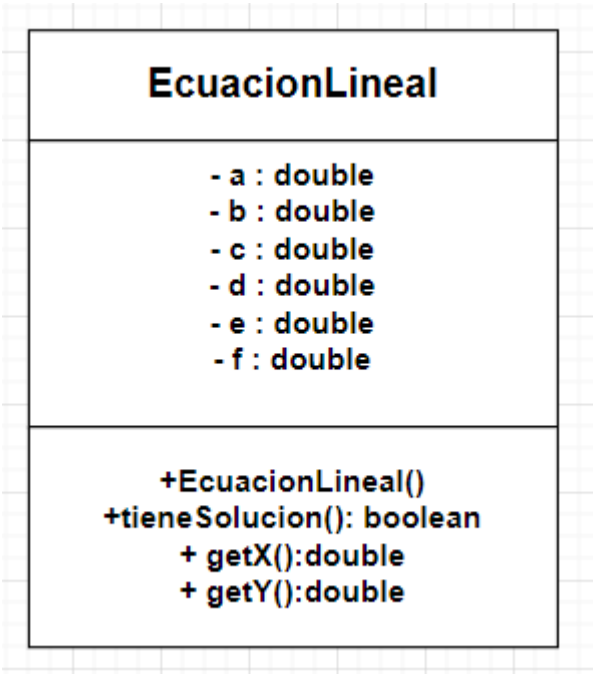
Ingrese a, b, c, d, e, f: `9.0 4.0 3.0 -5.0 -6.0 -21.0`

Ingrese a, b, c, d, e, f: `1.0 2.0 2.0 4.0 4.0 5.0`

Ejemplo de salida

```
x = -2.0, y = 3.0
La ecuación no tiene solución
```

//DIAGRAMA UML



//CODIGO JAVA

```
package ecu;

import java.util.Scanner;

import java.util.Locale;


class EcuacionLineal {

	//a) Los atributos privados a, b, c, d , e y f.


	private double a;

	private double b;

	private double c;

	private double d;

	private double e;

	private double f;


	//b) Un constructor para los argumentos  a, b, c, d , e y f.


	public EcuacionLineal() {

		Scanner sc = new Scanner(System.in);

		sc.useLocale(Locale.US);

		System.out.print("Ingrese a, b, c, d, e, f: ");

		this.a = sc.nextDouble();

		this.b = sc.nextDouble();

		this.c = sc.nextDouble();

		this.d = sc.nextDouble();

		this.e = sc.nextDouble();

		this.f = sc.nextDouble();

	}
```

```
//c) Un metodo tieneSolucion() que devuelve true si ad − bc no es cero

public boolean tieneSolucion() {
	double analizar = (a*d) - (b*c);
	if(analizar != 0) {
		return true;
	}else {
		return false;
	}
}

//d) Dos metodos getX() y getY() que retornan la solucion de la ecuacion.

public double getX() {
	double x = ((e*d) - (b*f))/((a*d) - (b*c));
	return x;
}

public double getY() {
	double y = ((a*f) - (e*c))/((a*d) - (b*c));
	return y;
}
}
```

```
//MAIN
package ecu;

class Main {

	public static void main(String[] args) {
		/*  Escriba un programa de prueba
			(Test) que solicite al usuario ingresar a, b, c, d, e y f
			y muestre el resultado. Si ad − bc es
			0, indique que ”La ecuacion no tiene solucion”.
		*/

		EcuacionLineal ec1 = new EcuacionLineal();



		if(ec1.tieneSolucion()) {
			System.out.println("X = "+ec1.getX()+", y = "+ec1.getY());
		}else {
			System.out.println("La ecuacion no tiene solucion");
		}

	}

}
```

// EJECUCION

```
Console
<terminated> Main (63) [Java Application] C:\Program Files\Java\jre1.8.0_471\bin\javaw.exe  (12 feb 2026, 15:52:41 – 15:52:43 elapsed: 0:00:01.815) [pid: 13616]
Ingrese a, b, c, d, e, f: 9.0 4.0 3.0 -5.0 -6.0 -21.0
X = -2.0, y = 3.0
```

3. (*Álgebra: Ecuaciones Cuadráticas*) Diseñe la clase *EcuacionLineal* para la ecuación cuadrática $ax^2 + bx + c = 0$. La clase contiene:

   a) Los atributos privados a, b y c, que representan a los 3 coeficientes.

   b) Un constructor para los argumentos para a, b y c.

   c) Un método **getDiscriminante()** que develve el discriminante $b^2 - 4ac$.

   d) Dos métodos **getRaiz1()** y **getRaiz2()** que retornan las dos raíces de la ecuación.

$$r1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \qquad y \qquad r2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Estos métodos solo son útiles si el discriminante es positivo. Si el discriminante es negativo, devuelva 0.

Dibuje el diagrama UML de la clase e impleméntela. Escriba un programa de prueba (Test) que solicite al usuario introducir los valores de a, b y c, y muestre el resultado según el discriminante. Si el discriminante es positivo, muestre las dos raíces. Si el discriminante es 0, muestre la única raíz. De lo contrario, muestre "La ecuación no tiene raíces reales".

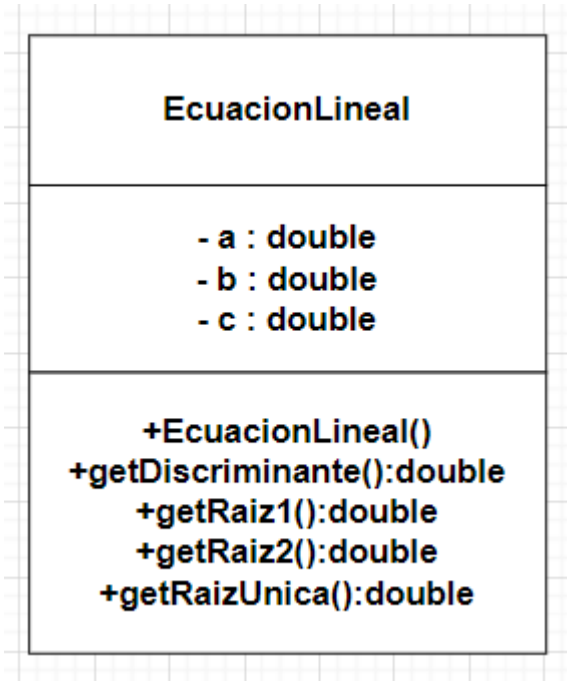
Ejemplo de entrada

Ingrese a, b, c: `1.0 3 1`
Ingrese a, b, c: `1 2.0 1`
Ingrese a, b, c: `1 2 3`

Ejemplo de salida

```
La ecuación tiene dos raíces -0.381966 y -2.61803
La ecuación tiene una raíz -1
La ecuación no tiene raíces reales
```

// DIAGRAMA UML



```
// CODIGO JAVA
package ecu2;
import java.util.Scanner;
import java.util.Locale;
import java.math.*;

class EcuacionLineal {

	//a) Los atributos privados a, b y c, que representan a los 3 coeficientes

	private double a;
	private double b;
	private double c;

	//b) Un constructor para los argumentos para a, b y c.
	public EcuacionLineal() {
		Scanner sc = new Scanner(System.in);
		sc.useLocale(Locale.US);
		System.out.print("Ingrese a, b, c: ");
		this.a = sc.nextDouble();
		this.b = sc.nextDouble();
		this.c = sc.nextDouble();
	}

	//c) Un metodo getDiscriminante() que develve el discriminante

	public double getDiscriminante() {
		double Discriminante = Math.pow(b, 2) - 4*a*c;
		return Discriminante;
	}
```

```
//d) Dos metodos getRaiz1() y getRaiz2() que retornan las dos raıces de la ecuacion

public double getRaiz1(){
        double d = getDiscriminante();
        double r1 = (- b + (Math.sqrt(d)) )/ (2*a) ;
        return r1;
}

public double getRaiz2(){
        double d = getDiscriminante();
        double r2 = (- b - (Math.sqrt(d)) ) / (2*a);
        return r2;
}

public double getRaizUnica() {
   return -b / (2*a);
}

}
```

```
// MAIN
package ecu2;

class Main {

	public static void main(String[] args) {
		EcuacionLineal ec = new EcuacionLineal();

		double d = ec.getDiscriminante();

		if (d > 0) {
		   System.out.println("La ecuacion tiene dos raices: "
			+ ec.getRaiz1() + " y "
			+ ec.getRaiz2());
		} else if (d == 0) {
		   System.out.println("La ecuacion tiene una raiz: "
			+ ec.getRaizUnica());
		} else {
		   System.out.println("La ecuacion no tiene raices reales");
		}

	}
}
// EJECUCION
```

```
Console
<terminated> Main (64) [Java Application] C:\Program Files\Java\jre1.8.0_471\bin\javaw.exe  (12 feb 2026, 16:07:37 – 16:07:39 elapsed: 0:00:01.810) [pid: 11460]
Ingrese a, b, c: 1.0 3 1
La ecuacion tiene dos raices: -0.3819660112501051 y -2.618033988749895
```

4. *Estadísticas: calcular el promedio y la desviación estándar.* En las aplicaciones comerciales, a menudo se le pide que calcule el promedio y la desviación estándar de los datos. La desviación estándar es una estadística que le indica cuán estrechamente se agrupan todos los diversos datos alrededor de la media en un conjunto de datos. Por ejemplo, ¿cuál es la edad promedio de los estudiantes en una clase? ¿Qué tan cerca están las edades? Si todos los estudiantes tienen la misma edad, la desviación es 0. Utilice la siguiente fórmula:

$$promedio = \frac{\sum_{i=1}^{n} x_i}{n} \qquad desviacion = \sqrt{\frac{\sum_{i=1}^{n}(x_i - promedio)^2}{n-1}}$$

Su programa debe contener los siguientes métodos:

- promedio() obtiene el promedio de los valores en punto flotante
- desviacion() devuelve la desviación estándar de los valores en punto flotante.

Escriba un programa que solicite al usuario ingresar 10 números, muestre el promedio y las desviaciones estándar de estos números.

Ejemplo de entrada

```
1.9 2.5 3.7 2 1 6 3 4 5 2
```

Ejemplo de salida

```
El promedio es 3.11
La desviación estandard es 1.55738
```

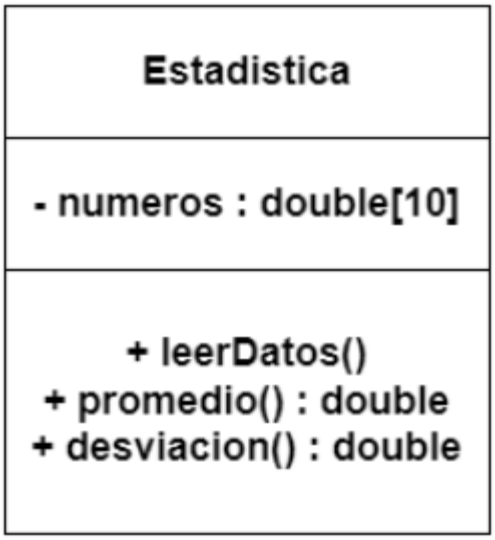
Obtenga la solución en la Programación Modular-Estructura y luego en Programación Orientada a Objetos, para ello dibuje el diagrama UML para la clase Estadistica y luego implemente la clase. Agregue de forma escrita las ventajas de utilizar la P.O.O.

// CODIGO JAVA – PROGRAMACION MODULAR ESTRUCTURADA

```
package est1;

import java.util.Scanner;

import java.util.Locale;


public class EstadisticaModular {

   public static double promedio(double[] numeros) {

      double suma = 0;

      for (int i = 0; i < numeros.length; i++) {

         suma += numeros[i];

      }

      return suma / numeros.length;

   }
```

```
    public static double desviacion(double[] numeros) {
        double prom = promedio(numeros);
        double suma = 0;

        for (int i = 0; i < numeros.length; i++) {
            suma += Math.pow(numeros[i] - prom, 2);
        }
        return Math.sqrt(suma / (numeros.length - 1));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        sc.useLocale(Locale.US);

        double[] numeros = new double[10];

        System.out.println("Ingrese 10 numeros:");

        for (int i = 0; i < 10; i++) {
            numeros[i] = sc.nextDouble();
        }

        double prom = promedio(numeros);
        double desv = desviacion(numeros);

        System.out.println("El promedio es: " + prom);
        System.out.println("La desviacion estandar es: " + desv);
        sc.close();
    }
}
```

//DIAGRAMA UML



// CODIGO JAVA – PROGRAMACION ORIENTADO A OBJETOS (POO)

```
package est2;

import java.util.Scanner;
import java.util.Locale;

class Estadistica {

    private double[] numeros = new double[10];

    public void leerDatos() {
        Scanner sc = new Scanner(System.in);
        sc.useLocale(Locale.US);
        System.out.println("Ingrese 10 numeros:");
        for (int i = 0; i < 10; i++) {
            numeros[i] = sc.nextDouble();
        }
    }
```

```
    public double promedio() {
        double suma = 0;

        for (int i = 0; i < numeros.length; i++) {
            suma += numeros[i];
        }
        return suma / numeros.length;
    }

    public double desviacion() {
        double prom = promedio();
        double suma = 0;

        for (int i = 0; i < numeros.length; i++) {
            suma += Math.pow(numeros[i] - prom, 2);
        }

        return Math.sqrt(suma / (numeros.length - 1));
    }
}

// MAIN
package est2;
public class Main {
    public static void main(String[] args) {

        Estadistica e = new Estadistica();

        e.leerDatos();
```

```
        System.out.println("El promedio es: " + e.promedio());
        System.out.println("La desviacion estandar es: " + e.desviacion());
    }
}
```

//EJECUCION

```
Console
<terminated> Main (65) [Java Application] C:\Program Files\Java\jre1.8.0_471\bin\javaw.exe (12 feb 2026, 19:01:57 – 19:01:58 elapsed: 0:00:01.624) [pid: 2836]
Ingrese 10 numeros:
1.9 2.5 3.7 2 1 6 3 4 5 2
El promedio es: 3.1100000000000003
La desviacion estandar es: 1.5573838462127583
```

Ventajas de utilizar la Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO) presenta diversas ventajas en el desarrollo de software. Entre las principales se encuentran:

1. **Organización del código:**
   Permite agrupar datos y métodos dentro de una misma clase, haciendo que el programa sea más ordenado y fácil de entender.
2. **Encapsulamiento:**
   Protege los datos internos de una clase y evita que sean modificados directamente desde el exterior, aumentando la seguridad y el control del programa.
3. **Reutilización de código:**
   Mediante la herencia y la creación de clases reutilizables, se puede evitar repetir código y facilitar el desarrollo de nuevos programas.
4. **Mantenimiento más sencillo:**
   Si se necesita realizar una modificación, esta puede hacerse dentro de la clase correspondiente sin afectar todo el programa.
5. **Mayor claridad y modelado del mundo real:**
   La POO permite representar objetos de la vida real dentro del programa, facilitando la comprensión del problema y su solución.
6. **Escalabilidad:**
   Facilita ampliar el sistema agregando nuevas clases o funcionalidades sin alterar significativamente la estructura existente.